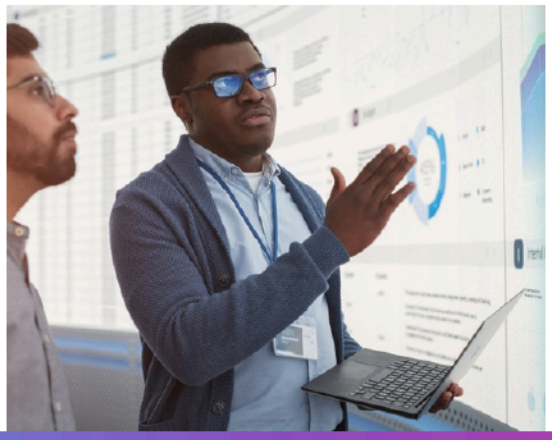



A CAREER GUIDE FROM CaRCC

Onramps to the RCD Field

A guide to careers in Research Computing and Data

Developed by the research computing and data community through the RCD Nexus Day workshop series, held in conjunction with the PEARC conference series, with support from Internet2

CaRCC
Campus Research Computing Consortium

Onramps to the RCD Field

A guide to careers in Research Computing and Data

Contents

Introduction: An Exciting Field Essential to Modern Scientific Research	**2**
Section One: What Is Research Computing and Data Work?	**4**
Section Two: Why This Work Matters	**9**
Section Three: What You Actually Need to Know	**12**
Section Four: What You Should Study to Work in RCD	**17**
Section Five: Where to Find a Job in RCD	**22**
Section Six: Translating What You Have Done Into What They Are Asking For	**26**
Section Seven: Meeting People, and Interviewing for an RCD Job	**30**
Section Eight: Where to Start	**35**
Appendix A: References and Resources	**37**
Appendix B: Acknowledgements	**41**

Introduction: An Exciting Field Essential to Modern Scientific Research

Think of a piece of research that has touched your life. A cancer treatment. A weather forecast that gave your town a few extra hours of warning. A vaccine that arrived faster than anyone expected.

Somewhere behind each of those results, it's likely that someone who is **not** a scientist made the computing work possible. Someone built and ran the machines that did the calculations. Someone organized the data so it could be trusted, shared, and used again. Someone made the software run correctly, and fast, and on a dataset far too large for any ordinary computer. More than 5,000 people in the United States work in this field. Modern science could not advance as rapidly without them, yet most people do not know the field exists.

It is called Research Computing and Data, usually shortened to RCD. Other terms that reference the same profession that you might encounter or be familiar with include advanced computing, high performance computing (HPC), cyberinfrastructure, supercomputing, and research IT. It covers the people who build and run the enormous computers that research depends on, who organize research data so it can be trusted and reused, who write and fix scientific software, and who sit down with a scientist and work out how to actually get an analysis done. If you have never encountered the term, don't be surprised. The field is genuinely hard to identify from the outside, and the people in it are the first to say so.

Who this guide is for

If you are considering this work. This guide is for anyone curious about or considering this work. Perhaps you are a student with a technical interest but no set plan. Maybe you are a researcher who has noticed you enjoy the computing side as much as or more than the science. You might work in IT (information technology) and want your work to have a larger societal impact. Or you could be a librarian, a lab technician, a teacher, or someone in an unrelated job who keeps ending up as the one who fixes the computer, untangles the spreadsheet, or figures out the new software. This guide is written for you first, and it assumes you know nothing about the field.

If you advise students or job-seekers. Academic advisors, career counselors, and campus career center staff regularly meet students whose skills point straight at this profession, and yet have no vocabulary for naming it. This guide supplies that vocabulary: what the work is, what it is called in job postings, which free programs students can join immediately, and how to recognize a fit.

If you are building programs. Faculty and administrators designing courses, certificates, student employment programs, or internship pipelines will find a description of what the field actually needs, which is not always what curricula assume.

If you already do this work. You may never have seen it described from the outside. Much of this is also useful for writing job postings and recruiting.

Where this guide came from

This guide was created by the people who do RCD work. Nearly 70 RCD professionals gathered for a workshop hosted by the Campus Research Computing Consortium (CaRCC) with a goal to answer a question the field had been circling for years: how does anyone find their way into this profession, and what would we tell them if we could?

They came from large research universities and small colleges, from libraries, national laboratories, and industry, and from every career stage. Working in small groups, they mapped out what the work actually involves, which skills genuinely matter and which are wrongly assumed to be required, where the jobs are hidden, and how people really got hired. What follows is their collective answer, which is why it may at times read like advice from someone who has been doing this for many years and wants to share what they learned.

How to read it

Read it in order or skip to what you would like to explore. The contents at the front will get you to the right place. If you would like to get started right away, Section Eight lists three free things you can do this month. Many sections end with suggestions for things you can do, and the appendix collects every resource mentioned in one list.

Section One: What Is Research Computing and Data Work?

The people who help make modern science possible, and how to become one of them

RCD professionals build and maintain research computing infrastructure, match scientists to resources they did not know existed, teach them to use those resources effectively, and find technology solutions for their complex research challenges. They are experts in the technology specific to research, and may specialize in areas such as computational systems, artificial intelligence, data, networking, software, and information security.

The field grew out of high performance computing, usually shortened to HPC. HPC means using far more computing power than any single computer can provide, to solve problems that would otherwise take months or years. That power usually comes from a cluster: hundreds or thousands of computers linked together so they can split up one large job, or run many smaller jobs at once, as a single system. A supercomputer is a very large, very fast cluster. The scope of the field now includes artificial intelligence, cloud computing, data management, refinement, and movement; research data compliance and security; training; and direct engagement with researchers to support them in using these technologies and avoiding pitfalls.



An RCD professional works through a computing problem with a researcher. Building a customized solution together, rather than handing over a standard service, is often what sets RCD apart from regular IT.

Why the job is hard to describe

Ask ten RCD professionals what they do and you will get ten different answers. That is the reality of this field. And because it is a relatively young and growing profession, specific RCD degree programs largely do not yet exist.

1. It sometimes looks like regular IT or computer engineering, but the work is very different. Enterprise IT delivers standardized services, like WiFi access and email, at a large scale for a higher education institution. RCD co-creates tailored solutions, with the RCD professional and the researcher building the computational method together and adapting it to a specific scientific question. Section Five explains how to spot the difference in a job posting.

2. The job changes by institution. At a large university it may mean working as part of a team operating supercomputers and managing massive data storage resources. At a small college it may mean one person knowing a bit about all aspects of the RCD profession and helping scientists access and use computational resources appropriate for their research. At a pharmaceutical company it may mean managing research environments for hundreds of scientists working on similar but uniquely challenging problems.

3. The best work is invisible within the research products. When RCD professionals do their jobs well, the researcher uses the system easily, gets what they need quickly, and publishes their findings without giving much thought to the technology that made it happen. The work is recognized by other RCD and research enablement communities, but most others will never know this work happens.

What you actually do all day

Here are some examples of actual roles, described by the people doing them. Common threads across these roles include: listening, problem-solving, helping others, and continuous learning. Staying current with new technology is both a requirement of the job and one of its real perks.

Title	A typical day
Senior Data Scientist	Research application support, building web tools that let researchers reach the supercomputer through a browser, tutorials and teaching, methodology review, automation, code optimization
Associate Director, Research Computing	Acting as the glue between what researchers want to do and what resources exist; matching people to technology they did not know was available
Director of Research IT	Identifying useful research technologies, designing a menu of services, onboarding researchers to shared national resources, operating a small campus computing cluster, a great deal of hand-holding
Senior Research Scientist	Teaching classes, supporting instructional and regulated-data research efforts, direct user support
Director, Research Computing and Digital Strategy	Supervising a team of eight research facilitators, fielding rising AI demand, and guest lecturing on topics like version control and Python
Industry Engagement Lead (Private Industry)	Building and deploying infrastructure that gives scientists access to computational resources, in partnership with a central research computing team
Librarian	Reference consultations with faculty and students, cataloging metadata, navigating open-access publishing questions, liaising with departments on emerging needs
Help Desk Support Staff	Responding to help tickets on login and access issues, escalating technical issues to specialized staff, updating documentation and FAQs

If you want to see how these roles are formally defined, the CaRCC HR Job Family Matrix was written for HR departments but can be useful to you. It lays out RCD position levels from entry to expert, with responsibilities and salary ranges, so you can see where you

might reasonably start. The median salary for mid-career professionals working in RCD was $86,000 according to CaRCC's 2025 Workforce Survey.

Two stories that make it concrete

Scaling up data analysis. A research team had their analysis working on a laptop, but only on a small sample of their data. The real dataset was a hundred times bigger. Moving to the campus supercomputer is not as simple as copying the files over, so a Research Facilitator met with the team to understand what their analysis needed, then reshaped it to run across hundreds of machines at once. The full-scale results became a leading publication in their field.

Making code restartable. A bioinformatics group comparing mRNA sequences ran analyses that took days, and any interruption meant starting over from the beginning. A Systems Software Engineer rewrote the software so it saved its progress as it went and could pick up where it stopped. That small change let the team use leftover computing time instead of waiting in line, and they started getting answers ten times faster.

Where RCD people work

- **Universities and colleges:** central research IT departments, research computing centers, individual labs, libraries, academic departments
- **National laboratories and national-scale computing facilities**
- **Industry**, particularly pharmaceutical and life sciences, energy, technology vendors, automakers and machinery manufacturers, and finance
- **Nonprofits and government agencies** supporting research missions

Common career tracks

The RCD profession organizes itself around what we call "facings," meaning who or what you spend your days facing: the researchers, their data, the software, the computer systems, or the institution. The framework came out of a 2018 workshop funded by the National Science Foundation and now structures the CaRCC Tracks. Most people move among facings over a career, and many occupy more than one at a time.

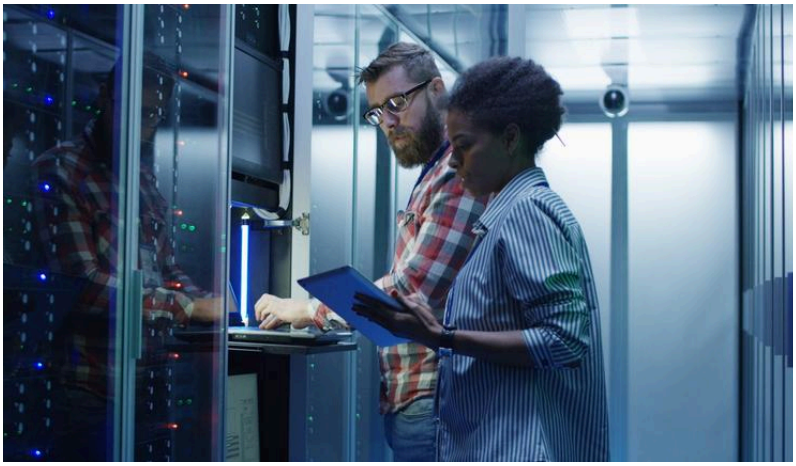
Researcher-Facing. You work directly with scientists, turning research questions into workable computational plans through consulting, training, and outreach. *Communication matters as much as technical skill***,** because much of the job is understanding what someone actually needs before any code gets written, or any resources are recommended.

Data-Facing. You help researchers *organize, move, store, publish, and visualize datasets*, which in modern research can be enormous. People arrive from any number of areas, including graduate research, general IT and systems administration, software engineering, science domains, libraries, teaching, and data curation.

Software-Facing. You write, build, install, and tune the software that research runs on, so that analyses finish in hours instead of weeks and work correctly on more than one machine at a time. Much of the craft lies in *making other people's code more efficient and*

more portable, while it can also involve being the primary software engineer writing code.

Systems-Facing. You *plan, build, and maintain the shared supercomputers themselves, along with the networks, data storage, information security, and cloud services around them*. When thousands of researchers depend on one system, keeping it running well is its own specialty.



Some RCD professionals set up and maintain supercomputers and computational systems. Others write software, work with research data, or develop RCD policies.

Strategy and Policy-Facing. You *set direction, build partnerships, pursue grant funding, and make the case to senior university administrators and government officials for the budget these systems require for construction and ongoing operation*. This is often the path into leadership, and it connects to peer organizations of research computing leaders and higher-education IT professionals.

How the facings work together. In practice, the facings operate as a team and many RCD jobs include more than one facing. Take the research team in the story above whose analysis outgrew a laptop. A researcher-facing facilitator met with them and reshaped the analysis. That work depended on systems-facing staff, who keep the cluster running; data-facing staff, who help move and store a dataset a hundred times larger than the original; and, often, software-facing engineers, who make sure the research code runs well across hundreds of machines. Strategy and policy-facing leaders made the case for the budget that bought the cluster in the first place. Each person sees a different part of the same problem, and the research succeeds because the pieces fit together. In smaller institutions, a similar situation might be managed by only one or two people who have cultivated skills in multiple facings.

The four career phases

The RCD community describes careers in four phases: pre-career, early, mid, and late. The phases are defined less by years or job titles than by scope: what you are responsible for and who relies on you. Because so many people enter RCD from other fields, your phase in RCD may not match your overall career stage. A lab manager with fifteen years in biology who moves into research facilitation is early career in RCD. This vocabulary matters when you look for training, since most professional development resources are labeled by phase.

Pre-career. You are exploring RCD but are not yet in an RCD role. This includes students, postdocs, researchers who've become the go-to computing person in their lab, and IT staff curious about research support. *Example: a graduate student who manages the group's computations and wonders whether that could be a job.*

Early career. You are in an RCD role and building skills, mostly working on defined tasks with guidance from others. This typically covers your first few years in the field. *Example: a facilitator or research software engineer who handles consultations, tickets, and workshops while learning local systems and research culture.*

Mid career. You work independently and own something: a service, a platform, a portfolio of research partnerships, or a small team. Colleagues and researchers come to you for answers. *Example: a lead facilitator who designs the campus training program, or a senior engineer responsible for a core system.*

Late career. Your impact is organizational or community-wide. You shape strategy, budgets, and hiring, and you contribute to the field through mentoring, governance, and national collaborations. *Example: a director of research computing, or someone leading a community organization or national program.*

These phases aren't a ladder everyone climbs at the same pace, and many people move between technical and leadership paths. Use them to find resources written for where you are now.

How people get in

There is no standard pipeline into the RCD field and the field highly values varied and non-traditional backgrounds. People arrive from graduate research, from enterprise IT and systems administration, from software engineering, from a specific science domain, and from libraries, teaching, and data curation to name a few. Section Four covers each of these routes and what to do from where you are standing.

Why people love the work

The recurring answer, in different words each time: you get to make other people's best work possible, solve unique and evolving challenges, create custom solutions, and work with like-minded people. You see things researchers do not. You bring capability back to your research institution. And regularly, someone tells you they had no idea that was possible.

Things you can do

Look through CaRCC's HR Job Family Matrix, which human resource offices can use to describe RCD roles and levels. As you read, notice which roles center on the parts of this work that appeal to you most, whether that's working closely with researchers, building systems, or connecting people to resources. That's a useful first clue about where you might fit.

Also consider asking one person who does this work what the best part of their job is. You'll likely hear some version of the answers above, plus details no guide can give you.

Section Two: Why This Work Matters

Because modern science cannot happen without it

This is a field where your work has visible consequences, and where there is often more demand for people than there are people.

Here are four reasons the work is worth doing.

1. You get to be the reason science happens better and faster

A typical example. A graduate student needs to run the same analysis on each of four thousand samples. She has been doing them one at a time, starting a run before she leaves at night and checking it the next morning. At that pace she will finish sometime next year. Someone from research computing shows her how to describe the task once and hand all four thousand copies to the shared supercomputer at the same time, since the supercomputer is built to run thousands of tasks at once. It finishes over one weekend.

Nothing about her science changed. Same analysis, same samples, same question. What changed is that she got a year of her life back and avoided many headaches along the way.

This is the shape of most RCD work. Someone is stuck and not sure where to go next. Their code is too slow, their data is too large, their software will not install, or their computing task keeps stopping or being interrupted halfway through. You figure out why, and then they are not stuck anymore. If you are the sort of person who enjoys that specific feeling, you will get it often.

What is unusual about it. You will not be the one whose name is on the research paper. Some people find that a dealbreaker, and that is worth being honest with yourself about. Others find it freeing: you get the satisfaction of the discovery without having to spend six years becoming a specialist in one narrow question. However, that doesn't mean your work will be unrecognized. Your solutions may be presented and shared at RCD professional conferences and within professional communities to benefit others working in the field.

2. You see more of the research world than almost anyone else in your institution

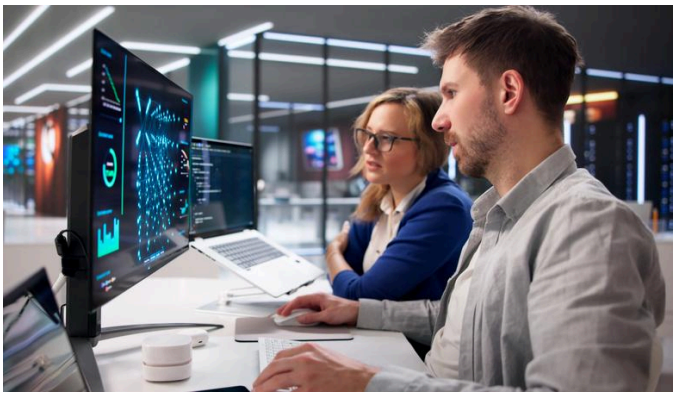
A real example. More than a thousand cameras sit on mountaintops and rooftops across California, turning in slow circles, watching for smoke. When they spot it, firefighters get an alert, sometimes before the first 911 call comes in. The detection system began as a research project at the University of California San Diego (UCSD) and has since been named one of *TIME*'s 2023 best inventions of the year.

The cameras are the visible part. The invisible part is the computing built to move, store, and make sense of an enormous and constant flood of images. RCD professionals at UCSD built that.

Why this matters for you. In a single week, one person in this field might help a geneticist studying a disease, a historian working with digitized archives, and an engineer running

simulations. You start to see how a problem solved in one field is the same problem someone in another field is still fighting. Carrying that solution across is a real contribution, and very few people on a university campus, or anywhere, are positioned to make it. Faculty are deep in their own disciplines by design. RCD professionals are usually a step removed from solving the core science problems.

It also means you are never bored in the way a single-subject job can make you bored. The topics, the problems, and the technologies all keep changing.



Behind every major research result is computing work most people never see. RCD professionals help scientists discover and use technologies that make research faster and easier.

3. The stakes are real, which is part of the appeal

A real example. A researcher wins a federal grant to study something sensitive, like patient health records, and the data comes with strict rules: who may see it, where it may be stored, who may log in to view it, and from where. The researcher is an expert in their science. They are not an expert in federal data security requirements, and they should not have to be.

Someone has to build a computing environment where that work can happen safely. Universities receiving significant federal research funding are increasingly required to certify that they run a formal research security program, including cybersecurity for sensitive research data. That is not paperwork somebody signs once. It means systems and practices are designed and operated in a particular way, by people who know how.

Why this matters for you. This is a job where you will have real impact right away. Whole research programs depend on the systems staying available, the data staying intact, and the rules being followed. That is real responsibility, handed to people relatively early in their careers, which is one reason the field tends to produce capable people quickly.

4. It is a career, not a stopover

This is a field and a community with an unusual amount of room to move. Because the work spans five different tracks and every institution divides it differently, a career here can change shape several times without changing employers. People move from supporting researchers to running systems, from systems into leadership, from a university to a national laboratory to industry and sometimes back again. The skills travel.

It rewards people who arrive sideways. If you have ever helped someone navigate data, wrangled a spreadsheet nobody else could fix, or been the person your lab asked when the software broke, you have already done a version of this work. Section Four covers how people get from where you are to here.

Things you can do

If you work at or attend a research institution, look for the RCD professionals already there. Start with the research computing or IT website, the library's research data services, or a faculty member who runs computationally heavy research and will know who supports their lab. The job titles listed earlier in this guide can help you recognize them. Then ask one of them about their job: how they got into it, what a typical week looks like, and how their role has changed over time. That last question speaks directly to this section, since most people in the field can describe at least one shift in the shape of their work.

If you are not at a research institution, the same conversation is possible through the community resources covered later in this guide.

Section Three: What You Actually Need to Know

The skills you should focus on developing

RCD work is most often done by teams. The long list of skills this field requires is a list for a *team*, not for one person. No individual has all of it, and nobody expects you to.

When RCD professionals were surveyed about their own jobs, only three percent said they spend their time in a single area of the work. Two thirds spend at least part of their time across four or more. That is what this field actually looks like from the inside: generalists who are deep in one or two things, working alongside other people who are deep in something else.

So the useful question is not "do I have the skills for this field." It is "what do I already bring, and what would I need to learn?" Almost everyone who does this work now had to learn something new.

Looking for training? The free and low-cost programs at the end of this section, under "Where to build these skills," are the fastest way to get started.

What is genuinely required of everyone

Listening well and asking good questions. Someone arrives at your desk and describes their problem badly, because they do not yet know what the problem is. Your first job is to ask questions until the real issue surfaces. This is the single most important skill in the field, and it is the one hiring managers most often say is missing.

Not giving up. Problems in this work are frequently unique and with undocumented processes. The people who succeed are the ones who keep going, get creative, and go find help rather than saying they cannot help.



Listening well comes first. Understanding what a researcher actually needs is the most important skill in RCD, and the technical skills can be learned along the way.

Being willing to learn continuously. Technology and science change. You will be a beginner at something new every year for your entire career. If that sounds exciting rather than exhausting, this might be a great career for you.

Explaining things clearly, in writing and out loud. You will write documentation, describe a technical constraint to a professor, and translate a researcher's question into something a computer can do. Clear writing is a technical skill here.

Humility. You will not always know the answer. You will sometimes be wrong or unsure in front of people who are counting

on you. Saying “I made a mistake; let’s try another approach” or “I don’t know, let me find out and get back to you” is a professional strength in this field, not an admission of failure.

Comfort in a research environment, or ability to adapt to one quickly. Not expertise in any one science, but a working sense of how research operates: that projects run for years, that grant deadlines are immovable, that a graduate student’s thesis timeline is a real constraint, that publication matters enormously to the people you serve. If you are coming from outside academia, this is the piece you will need to learn, and most people absorb it within a year.

If you have those six skills, the technical parts are learnable. The reverse is not true, which is why teams hire for these six skills.

Five things people wrongly assume they need

This is the part of this section worth reading twice, because these assumptions keep good candidates from applying.

“I need to know everything.” No. You need to know how to find out, and who to ask. Every RCD professional works surrounded by things they do not personally understand. The research facilitator role is built on this: much of the job is connecting researchers to the right resource or expert, not being that expert.

“I need a Ph.D.” No. Some roles are held by people with doctorates, often because they came from research, such as computational scientist. But many roles, including HPC systems administrator, research facilitator, and research data consultant, are regularly filled by people with a bachelor's degree or relevant experience instead. The field's own job framework runs from entry level through expert, and entry level means entry level.

“I need to be a specialist already.” No, and specializing too early can work against you. This field rewards people who can move among areas. Research data consultants, for example, often draw on statistics, data management, and a working knowledge of several research disciplines at once.

“I need to be a generalist who is decent at everything.” Also no. That is the opposite mistake. Bring real depth in something, then broaden. Look at how the more technical roles, such as research software engineer or HPC systems administrator, usually name a clear core competency. Depth is what gets you in the door.

“I need to have deep knowledge before I start.” This is the most damaging one, because it is exactly backwards. These are positions that many people grow into. Institutions often hire for basic technical knowledge paired with aptitude and interest, then train. If a posting reads as though it wants a finished expert, that may be what they are hoping for, but do not assume it automatically rules you out.

The technical side, honestly

You do need some technical foundation. Here is the honest version, separated into what is nearly universal and what depends on where you land. You can learn all four for free, and many people learn them on the job.

Close to universal, whatever role you take:

- Comfort at a command line, and basic familiarity with Linux, the operating system nearly all research computing runs on
- Some scripting or programming, enough to automate a repetitive task. Python and Bash are the most common starting points
- Version control, meaning tools like Git that track changes to code and let teams work on the same thing without destroying each other's work (although many RCD professionals learn this on the job)
- Troubleshooting as a habit of mind: forming a guess about why something broke, testing it, and narrowing down

Everything else depends on the track. Toward systems-facing work, you will learn cluster management, storage, networking, and cybersecurity. Toward data-facing, you will learn databases, metadata standards, data management planning, and how research data gets published and cited. Toward software-facing, you will learn development practices, testing, and how to make scientific code run correctly somewhere other than the computer it was written on. Toward researcher-facing work, you will learn scientific computing tools and enough of many domains to ask good questions.

How RCD work differs from regular IT

As Section One describes, RCD work differs from enterprise IT. People moving from enterprise IT into RCD may need time to adjust, not because the work is necessarily harder, but because it asks for a different way of approaching problems. The goals are less defined, the path forward may not be known, and success often depends on exploring the problem alongside the researcher. Section Five will help you spot these differences in job postings.

What a week can look like

For a researcher-facing team. Monday you help a graduate student figure out why her analysis crashes on the tenth of forty files. Tuesday you are in a planning meeting about next year's data storage purchase. Wednesday you write step-by-step instructions for a new tool, which nobody will read until they need it desperately. Thursday a professor asks whether the university can host data with federal handling restrictions, and you spend the afternoon finding out. Friday you teach a workshop to eleven people, four of whom have never used research computing of any kind.

For a systems-facing team. Monday you haul out old equipment and replace a failed disk drive at the data center. Tuesday you update firmware on the compute nodes during a maintenance window. Wednesday you get an alert on a temperature sensor that sends you back to the data center to investigate. Thursday you work with a hardware vendor on a GPU

that's been regularly failing. Friday you work with user support staff on a researcher's job that has routinely failed to start.

Some weeks are quieter. Some weeks a system fails and everything else waits.

The unwritten rules

Experienced people in this field are unusually direct about the professional norms, so here they are plainly.

- You will make mistakes, including consequential ones. Hiding a mistake is the actual failure. Say it early, fix it, and do not make that same one again.
- It is fine to say you will get back to someone. It is not fine to lose your patience with a person who came to you for help, and it is not fine to embarrass them.
- Never tell someone that what they are struggling with is easy. What is obvious to you likely took you years to absorb, and saying it is easy teaches them not to ask you again.
- When people may already be frustrated and slightly embarrassed about what they do not know, how you treat them determines whether they come back before the problem is a crisis.

Where to build these skills, free or low-cost

Most programs below are free or low-cost, and open to students and people not yet working in the field.

- **Campus Research Computing Consortium** (CaRCC). A free-to-join professional organization for RCD professionals that provides community and professional development. CaRCC welcomes students and aspiring RCD professionals, and is a great way to learn about the field and build connections.
- **Virtual Residency Program** (University of Oklahoma). A free summer workshop, run online, focused on the professional and interpersonal side of research computing facilitation, including how to talk to researchers. No prerequisites at all. It has reached more than 1,300 people from over 440 institutions.
- **CyberAmbassadors.** Training in communication, teamwork, and leadership built specifically for people supporting computational research. Free, offered virtually several times a year, with a certificate for completing the modules. Open to students.
- **Campus Champions.** Roughly 800 members at more than 300 institutions, connecting people who help researchers use computing resources. This isn't an organization you can join as a student or pre-professional, but if you are at a university, find out whether your campus already has a Champion and reach out to them to see if they can offer advice, support, or mentorship.
- **CCMNet and ACCESS MATCH.** Peer and student mentorship programs offered by groups that support shared tools and national computing resources for U.S. researchers.
- **Linux Clusters Institute workshops.** Five days, aimed at Linux administrators new to HPC, with hands-on lab work building a small test cluster. If systems work appeals

to you, this is the route to take. It is worth noting that these workshops do have a cost and require travel.

A reasonable first move. Pick the program closest to what already interests you, and join a CaRCC track call in the same month. Between the two you will learn more about whether you want this work than any amount of reading, including this guide.

Section Four: What You Should Study to Work in RCD

There is no such thing as a "right" path into RCD work

Almost nobody working in research computing today studied something called research computing. The degree doesn't exist. The people running university supercomputers, curating research data, and helping scientists scale up their work arrived from physics, from libraries, from IT help desks, from postdoctoral work, from undergraduate studies, from industry, and from a few years of quietly being the person in a research lab who could fix things.

This has a practical consequence worth stating plainly: if you are looking for the credential that qualifies you for this field, you will not find it, because there is not one. No license, no required certification, no gatekeeping exam. What gets people hired is evidence that they can do the work and have *interest in doing it*.

That means the path into RCD work where you are standing right now is shorter than you might think.

Degrees that align, including the ones you would not guess

The traditional ones. Computer science, information science, data science, computational chemistry, bioinformatics, physics, astronomy, meteorology, statistics. If your degree involved writing or running code to analyze data or solve research problems, you are already partway in.

The useful measure. Any field that requires more computing power, larger data sets, or specialized hardware than a laptop can provide is a field that produces RCD professionals. That is the actual test, and it is broader than people expect.

The ones that surprise people. Archaeology, which now runs on lidar (laser-based 3D mapping) and 3D scanning. Digital humanities and history, working with millions of scanned pages. Social science, with survey and administrative data at national scale. Sports analytics. Music and media, with large audio and video collections. Library and information science, which is a major and well-established entry route into data-facing work. Public health. Environmental science. Economics.

And no degree at all. Community college, vocational training, military service, and self-taught paths are real and common, particularly on the systems side, where what matters is demonstrated Linux competence rather than a diploma. However, degree-granting programs will give you an advantage by helping you build research expertise and experience working on collaborative projects.

Where your background might lead. These are tendencies, not rules, and many people move among facings over a career. Data science, statistics, and library and information science often lead toward data-facing work. Computer science and software engineering often lead toward software-facing work. Experience in IT, networking, or systems administration often leads toward systems-facing work. A degree in a science field, especially combined with teaching or tutoring experience, often leads toward

researcher-facing work. Strategy and policy-facing roles usually come later, after experience in one of the other facings.

Important note: If your field is not listed, that is not a verdict. Ask instead whether people in your field of work or study struggle with data or computation. If they do, you have a way in.

Routes in, depending on where you are standing

If you are in high school or community college. Look for a campus computing center at any nearby university and ask whether they hire students or take interns. Learn Linux and the command line now, because it is the one skill that transfers everywhere in this field. The Student Cluster Competition at the annual Supercomputing (SC) conference is open to high school and undergraduate students, and there is a virtual companion competition designed for teams without much funding or prior experience.

If you are an undergraduate or graduate student. You have more access than you probably realize: a campus computing group that will give you an account, training you can take for free, and faculty who need computational help. The list below is written for you.



There is no single degree that leads to a career in RCD. Professionals arrive from a wide range of academic majors, prior careers, and military service.

If you are a postdoc, or leaving the faculty track. Your research experience is valuable. It is the specific thing this field is short of. You already know how research works, how grant cycles feel, what a graduate student is up against, and how to read a paper in an unfamiliar field. That knowledge takes years to acquire and cannot be trained quickly. What you may need to add is breadth across tools and comfort supporting other people's work rather than your own. People make this move constantly, in both directions.

If you are a librarian. You are closer to this field than your job title may suggest. Data management plans, persistent identifiers, and digital preservation are core RCD needs. Start with whether your campus computing department has an existing relationship with the library's services. If they don't, this could be your opening.

If you work in campus IT already. This route is well worn. Your Linux, networking, storage, and security skills transfer directly, and the adjustment is more cultural than technical, as Section Three describes.

If you are in industry, a medical research hospital, or a national lab. You likely already have adjacent skills, and possibly stronger engineering practice than the academic norm. What you may be trading, in some cases, is compensation for autonomy, variety, and mission.

If you are changing careers from something else entirely. Start by learning the vocabulary, because the barrier is more often language than aptitude. Then find any excuse to do applied work, however small. The free resources below are where to begin.

If you are still a student, start here

This is the most concrete advice in this guide, and it comes from people who hire. You do not need to do all of these, and you do not need technical skills to begin. Start with whichever feels most manageable. If terms like "cluster" are new to you, that is normal; the training programs in Section Three teach them from the beginning.

Get close to research. Ask a professor whose work interests you whether you can help in their lab, or look for undergraduate research opportunities on your campus. Being part of a research project shows you the computing problems researchers face, and it makes every other step on this list easier.

Move your work off your laptop. If a class has you running analyses locally, run the same thing on your campus computing cluster instead. Ask your research computing group for an account. If your campus has nothing, U.S. graduate students can request free time on national supercomputers through NSF ACCESS, and the smallest allocation of time takes only a few sentences to request. Undergraduate students can access these resources if they work on a research project under an advisor. Having done this once puts you ahead of most applicants.

Do more computation than the assignment requires. Automate the tedious part. Scale the dataset up. Make it reproducible. Run it on several different computers. Implement checkpointing (saving a program's progress as it runs, so it can restart where it left off) on long-running programs. The habit is the point, and it is visible to anyone reading your resume.

Take whatever training your institution already offers. Cluster computing, data management, data security, HPC introductions. Take them even if the topic seems unrelated to your plans. You are there to learn the language, the terminology, and the culture, so that when you sit in a room with researchers you can follow the conversation.

Present something. Write a poster about work you did and submit it to a regional or national research computing event, or one at your own institution. It is more accessible than people assume, it puts you in a room with people who hire, and it demonstrates exactly the mix of technical and communication skills this field wants.

One more, if you can. Join or start a student cluster competition team. Teams design, build, and tune a small supercomputer and compete against real scientific workloads. It is the most direct hands-on introduction to systems work that exists, and alumni turn up throughout this field.

For more student entry points, including campus jobs, summer research programs, and national laboratory internships, see "Ways to get started" in Section Five.

About credentials, honestly

What does not exist. As mentioned, there is no single certification that makes you an RCD professional. No professional body licenses this work. Nobody is formally checking your credentials.

They can be limiting. Vendor certifications are the credentials most visible to newcomers, and they are the least aligned with this field. A Microsoft certification signals enterprise IT rather than research computing. Cloud vendor certifications are more mixed, since some RCD roles are genuinely cloud-heavy, but they are not the currency here.

What actually functions as a credential. Evidence that you have done the work: a cluster account you have used, code in a public repository, a poster you presented, a workshop you helped teach, a certificate from one of the community training programs in Section Three. Visible participation in the professional community counts more than most formal credentials, because this is a field where people hire people they have seen ask a good question in a community forum.

The reframe. If you are worried about being gatekept by credentials, the honest news is that the gate is mostly staffed by people who were let in the same way. Confidence and vocabulary are the real barriers, and both are learnable in months.

Free things to learn from

Section Three lists the community's own training programs. These are additional, and mostly self-paced.

- **The Carpentries.** Hands-on workshops teaching the foundational skills for working with data and code, aimed explicitly at novices with little or no prior experience. Four curricula: Software Carpentry for coding practice, Data Carpentry for working with research data, HPC Carpentry for working on a computing cluster, and Library Carpentry for people in library and information roles. Lessons are freely available online, and workshops are taught by trained volunteers worldwide. Many universities run them, so check your own campus first.
- **The Turing Way.** A free, community-written online handbook on reproducible, ethical, and collaborative research. It is not aimed at supercomputing specifically, but its chapters on reproducible research, project design, collaboration, and communication cover ground that no course teaches and that this field expects you to know. Not meant to be read cover to cover. Start with whatever you need now.

- **The RCD Ecosystem table.** A community-maintained map of the organizations in this field, which is otherwise genuinely difficult to navigate from outside. Useful for working out who does what and where you might belong.
- **Learning how science works.** If you are coming from a technical background without research experience, the gap is usually in understanding the scientific process itself: how a hypothesis becomes a study, why methods matter, how peer review functions. Visionlearning's free modules on the process and practice of science are a reasonable starting point.
- **LinkedIn Learning.** Many institutions hold a site license, which makes a large catalog of technical courses free to you. Worth checking whether yours does before paying for anything.
- **Cloud, GPU, and AI training.** Major vendors (e.g. NVIDIA, Microsoft Azure, AWS) offer substantial free introductory material, and it is genuinely useful for AI-adjacent research support. Take the training and skip the certification, per the section above.

One caution

Several experienced people in this field want you to know the following:

Being proactive is how you succeed in this work. You see a gap, you fill it. Nobody asked you to, and it turns out to matter. That instinct is the single most valuable trait a new RCD professional can have.

It is also how people burn out. The gaps are infinite, the staffing is usually thin, and researchers are unfailingly grateful in ways that make it hard to stop. Learn early which gaps are yours to fill and which belong to someone else. The people who last in this field are the ones who figured that out sooner rather than later.

Section Five: Where to Find a Job in RCD

Normal job searching does not work for this field

If you type "research computing jobs" into a job site, you will find very little, and you will conclude the field is not hiring. It is hiring. You are looking in the wrong places, and that is not your fault.

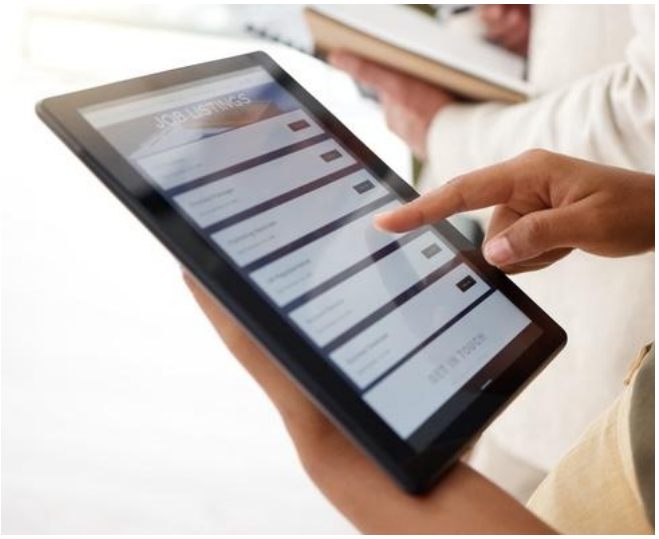
Here is why. Universities have small advertising budgets, so most positions get posted only on the institution's own careers page and then circulated by the hiring manager through professional mailing lists and Slack channels. Mailing lists do not appear in search results. If you are not already on the list, the job is effectively invisible to you. On top of that, job titles in this field are assigned by human resources departments using classification systems that often have nothing to do with the actual work.

The practical consequence: finding these jobs is less about searching harder and more about being in the right rooms and knowing what the postings actually look like. Both are fixable in an afternoon.

The most useful skill: reading past the job title

Titles mislead. Not deliberately. A position doing exactly the work described in this guide might be titled Systems Analyst III, Computational Scientist, Research Programmer, IT Specialist, or Senior Scientist. Renaming a job classification at a university is a bureaucratic ordeal, so many groups advertise under whatever title their HR system permits and describe the real job in the body of the posting.

So read the description, every time. The signal is in the duties, not the header.



RCD jobs often hide behind titles like Systems Analyst or IT Specialist. The best way to find one is to read the job description, not just the title.

Words that indicate an RCD job, wherever they appear in the posting: research computing, research computing and data, RCD, HPC, high performance computing, advanced research computing, cyberinfrastructure, research data, scientific computing, research facilitation, research software engineering, research IT. Also look for mentions of working with faculty, labs, graduate students, grants, or specific sciences.

Job titles that don't obviously read as RCD but often are: Computational Scientist, Application Scientist (common at national laboratories for researcher-facing roles), Cyberinfrastructure Engineer, and titles built on "research software." If a title looks unfamiliar, check the posting text for the words above.

Words that suggest it is regular IT instead: a vendor certification as the central qualification. A posting built around Microsoft certification is almost certainly enterprise IT. Cloud vendor certifications are more mixed, since some genuine RCD roles are cloud-heavy, so read further before ruling it out.

Do not self-eliminate. This is the most common mistake newcomers make. If a posting mentions supporting genomics research and you know nothing about genomics, apply anyway. Domain expertise is almost never a real requirement in these roles, and postings that name one discipline usually do so because that is where the funding came from, not because they will turn away everyone else. Section Three explains why teams hire for aptitude rather than existing knowledge.

Where to actually look

Community channels, which are where the real ones are

These are volunteer-run and unglamorous, and they carry postings you will not find anywhere else.

- **Ask.CI jobs category.** A question-and-answer site for people who do research computing, with a section specifically for job postings. Publicly readable, no membership needed.
- **US-RSE job board.** The U.S. Research Software Engineer Association maintains an actively updated, browsable list of research software positions across universities, national labs, institutes, and industry. Probably the best-maintained board in this space.
- **CaRCC People Network.** Free to join. Postings circulate on the track email lists and in the community Slack, so you need to be subscribed to see them.
- **ACM SIGHPC Systems Professionals.** For systems-facing work. The chapter runs a Slack workspace where positions get shared.
- **Code4Lib jobs.** For data-facing and library-adjacent roles.
- **Research Data Access and Preservation Association.** Another route into data-facing work. Jobs are often promoted over their mailing lists.
- **Reddit.** The HPC and scientific computing communities carry occasional postings and a lot of candid discussion about the work.
- **Digital Research Alliance of Canada.** If you are looking in Canada.
- **Women in HPC Job Board**. For jobs in the USA, Canada, and Europe.

National aggregators

These organizations and websites can be a good place to search for jobs. Use the search terms above rather than browsing by category: HigherEdJobs, the Chronicle of Higher

Education, EDUCAUSE, HPCwire, Inside Higher Ed Careers, the Computing Research Association career center, jobRxiv, and LinkedIn.

Institutional pages, checked directly

Many jobs never leave the university's own site. Pick the ten institutions within reach of where you live or want to live, find their research computing group, and check their HR department's careers page every couple of weeks. Also just email the group. A short note asking whether they anticipate openings, or take students, or would talk to you for fifteen minutes about the work is normal and welcome in this community.

National laboratories

Sandia, Oak Ridge, Pacific Northwest, Argonne, Los Alamos, Lawrence Berkeley, and others all hire heavily for this work, post on their own sites, and run substantial internship programs. Each has its own process, so go to the individual lab. The National Laboratories Career Page also lists jobs across all the labs.

Conference career fairs

The SC, PEARC, Tapia Diversity in Computing, and Grace Hopper Celebration of Women in Computing conferences all run job fairs where dozens of employers show up specifically looking for people in this field. If you can get to one, it compresses months of searching into a few days.

Ways to get started

If you are a student, these are the entry points:

Campus student jobs. The most direct route. Ask your university's research computing group whether they hire students, even if nothing is posted.

NSF Research Experiences for Undergraduates. You apply directly to individual sites, which host you for a summer of research, usually about ten weeks, with a stipend and often housing and travel support. Crucially, you can apply to sites at universities you do not attend, which makes this available to students whose own campus has little research computing.

National laboratory internships. Competitive, well-paid, and a strong signal on a resume.

Student cluster competitions. Covered in Section Four. Advisors often hire from their own teams.

Your own campus career center. They may not know the phrase "research computing," so bring the search terms from earlier in this section and ask them to help you look.

An honest word about first jobs

Many people's first foothold in this field is a help desk or user support role. That is not what most graduates picture, and it can feel like a step sideways.

Two things worth knowing. First, it is a genuinely good way in, because supporting users teaches you the systems, the tools, and the researchers all at once, and internal hiring from support into facilitation or systems work is common. Second, this is not unique to research computing. New employees in most professions spend their first year or two doing the less glamorous version of the job. The difference here is that the ladder above the support role is unusually short. Along the way you meet plenty of people and gain insight into the breadth of the field.

If you take one of these roles, ask on day one what the path up from it looks like. Groups that have thought about this will have an answer.

For hiring managers and anyone writing a posting

We included this because the people who read this guide are also, eventually, the people who will write these postings.

Use the community's common terms somewhere in the posting, even parenthetically, even if HR controls your title. If the working title is Systems Analyst III, put "research computing" and "HPC" in the duties section so the posting is findable. Name the disciplines the role will support, and state explicitly whether domain expertise is expected, because it usually is not and candidates assume otherwise. Say plainly that people from non-traditional backgrounds are encouraged to apply. Then post it somewhere findable, not only to a mailing list.

Vague postings do not get filled. Specific ones do.

Section Six: Translating What You Have Done Into What They Are Asking For

How your existing skills align with RCD jobs

There is a particular failure that happens over and over in this field. A hiring manager writes a posting for someone to help researchers use a supercomputer. Somewhere out there is a person who spent two years keeping a lab's analysis pipeline running and would be excellent at that job. Neither one recognizes the other, because they do not share a vocabulary.

The hiring side writes in terms like research computing, cyberinfrastructure, HPC, research facilitation, and research software engineering. The person looking is more likely to be searching for words like science, lab, supercomputing, computer cluster, or research support. Those describe the same work. They just do not overlap as search terms.

Section Five covers where to look and how to read past a job title. This section is about the harder part: recognizing that your own experience already counts, and describing it in language a hiring manager will register.

Translating your experience into theirs

The move is to stop describing *what you did* and start describing *what capability it demonstrates*. Your work does not need to have happened in a research computing group to count.

What you actually did	**How to describe it**
Tutored students in math or coding	User training and technical instruction
Worked an IT customer service or retail job	User support; translating between technical and non-technical audiences
Kept your lab's data organized and backed up	Research data management
Ran analyses on a departmental server or cluster	Experience with shared computing environments and scheduling computing jobs
Wrote scripts to clean or move data	Data pipeline and workflow development
Mentored younger students in a lab	User training and technical support, mentoring, peer mentoring
Wrote up procedures so others could repeat your work	Technical documentation
Worked an IT help desk	Technical support and troubleshooting in a service-oriented role
Installed or troubleshot scientific software for yourself or others	Scientific software support

What you actually did	How to describe it
Made explanatory videos, tutorials, or slide decks	Training material development, technical documentation, and technical communication
Built or maintained a website for a lab or project	Web development supporting research
Learned a new tool because a project needed it	Continuous learning and adaptability, the most requested trait in this field
Figured out what someone actually needed when their question was unclear	Requirements gathering and consultation

The most undervalued item on that list is teaching. Formal or informal, paid or not. A large share of RCD positions involve training somebody at some point, and postings rarely say so explicitly. If you have taught anything, say so.

The second most undervalued is Linux. Any command line experience at all is worth putting on the page. It is the closest thing this field has to a universal requirement.



Customer service, troubleshooting, tutoring, and keeping a lab's data organized all count. Many RCD skills start in jobs and classes that have nothing to do with supercomputers.

Where translation becomes overclaiming

Being generous with your own experience is fine. Stretching is not, and hiring managers in this field notice.

Audio-visual IT support is not research computing facilitation. Both involve helping people with technology, but the substance is different, and claiming the mapping suggests you do not understand what the job is. Similarly, mentoring peers is not supervision. Mentoring is genuinely valuable and worth listing as mentoring; calling it supervision invites a question you will not answer well.

The line is roughly this: name the capability honestly and let the reader judge whether it transfers. Do not assert a job title you have not held.

A practical exercise. Take one thing you have actually done and write two sentences about it: what the problem was, and what you did about it. Do that for four or five experiences. That is most of a resume for these roles, and it works better than a keyword list, because these are jobs where the hiring manager is trying to find out how you think.

Reading a posting closely

A good posting is specific about who you will help and what they are working on. In one example, a posting for an RCD professional at a coastal university's marine science center named the actual research it supports: genomics and bioinformatics, oceanographic and coastal modeling, computational models of chemical structures, video and photo processing, sensor development, and labs beginning to move into machine learning. A reader can tell what the job is, who it serves, and what they would learn.

A bad posting talks about stakeholders and solutions and says nothing concrete. Real examples of language that tells you almost nothing:

> "You will work across stakeholder groups to enable full use of the University's IT systems and services."
>
> "Proactively drive conversations with stakeholders to identify priorities for delivery of solutions and judicious experimentation with new concepts."

Both are real. The second could describe six unrelated jobs.

What to do with a vague posting. Do not discard it, because vagueness usually reflects an HR template rather than the actual role. Look for the tells: does it mention faculty, labs, grants, graduate students, or any named science? Does it list Linux, clusters, or scientific software? If so, email the contact and ask what a typical week looks like. Groups that struggle to describe their own openings are often relieved when someone asks, and a vague posting is frequently a sign that the position has been hard to fill.

A warning in the other direction. Some postings are clear and specific about being something other than what you want. A title like Software Performance Engineer, paired with a description built around one deep technical competency, is telling you the truth: it is a specialist role. Titles including help desk or general support are also telling you the truth: those are human-centered roles, which may be exactly what you want. Believe the specific postings.

What every one of these jobs wants, whether it says so or not

Across nearly all RCD postings, four things are actually being screened for even when they appear nowhere in the requirements:

- Genuine interest in science and research
- A service orientation, meaning satisfaction in other people's success
- Willingness to keep learning indefinitely
- In researcher-facing roles, the ability to work alongside scientists without being intimidated by them

Address these somewhere in your application. Most candidates do not, and it is the easiest way to stand out.

One thing about money that affects how you read a posting

Research computing positions are funded in different ways, and it matters.

Some are paid from an institution's regular budget, sometimes called "hard money", which makes them about as stable as any university staff job. Others are paid from grants, sometimes called "soft money", which means the position exists as long as the funding does. A grant-funded role might be excellent, well-paid, and lead somewhere, but it can also be term-limited in ways the posting does not spell out.

Institutions also differ enormously. A large research university may employ dozens of people across specialized teams, with a defined career ladder. A smaller college may have one or two people doing everything, which means broader work, faster learning, and a shorter path to real responsibility, but fewer people to learn from and less room to advance in place.

Neither is better. But it is a fair and normal question to ask in an interview: is this position funded from the operating budget or from a grant, and what happens at the end of the grant. Ask it and see where the conversation goes.

Section Seven: Meeting People, and Interviewing for an RCD Job

How to network, join the RCD community, and get the job you actually want

Section Five explained that most positions never appear anywhere findable. Here is the other half of that: they get filled through people who already know each other.

That may sound like bad news if you don't know anyone, but this community is small, unusually welcoming, and full of people who remember being new. It is easy to get to know someone in the field with just a little effort.

Something more surprising also happens regularly. A hiring manager meets someone capable, has no opening, and then writes a position around them when funding appears. Positions in this field are frequently shaped by who is available rather than the reverse. You cannot make that happen on demand, but you can be the person it happens to, and it only happens to people the community has met.

So networking here is not schmoozing. It is the actual distribution channel for opportunity, and it doubles as how you learn the work.

Where to meet people, starting from free

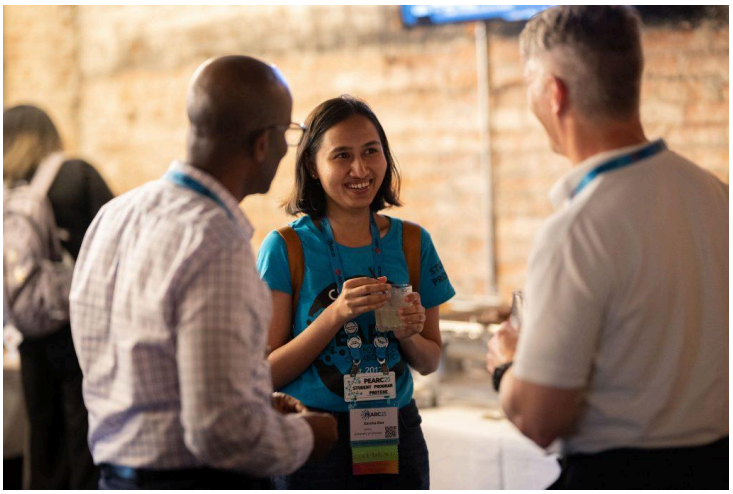
Online, free, this week

- **CaRCC People Network track calls.** Monthly, by area of interest, free to join. Show up and listen. Nobody will ask you to justify being there.
- **Ask.CI.** Publicly readable question-and-answer site. Reading it teaches you the vocabulary and the concerns of the field faster than anything else, and answering something eventually is how people learn your name.
- **Community Slack workspaces.** The ACM SIGHPC Systems Professionals chapter runs one. US-RSE runs one. These are where practitioners actually talk.
- **Mailing lists, including ones outside this field.** A useful trick: RCD opportunities and announcements often get cross-posted into disciplinary lists. If you are on a physics or biology or library list, watch for computing announcements arriving from outside. That is often the first thread someone follows into this community.

Conferences, which are the highest-value thing you can do

Two conferences matter most, and both have substantial programs specifically for newcomers.

PEARC (Practice and Experience in Advanced Research Computing) is the research computing and data community's own conference. Its student program has grown into a full week of programming, and the 2026 edition served 107 students. Crucially, it is designed for people with no background: open to students 18 and older, explicitly welcoming those who have never used high performance computing and who do not come from a technical major. The program includes mentorship, resume clinics, interviewing skills sessions, introductions to HPC, an internship spotlight, structured meetings with exhibitors who are recruiting, and sometimes a data center tour.



In a small, welcoming field, connections lead to opportunities. Conversations at conferences and community events such as PEARC and SC are often how people hear about their next job. (Photo courtesy of PEARC Conference Series.)

SC is the large international supercomputing conference. Its Students@SC program includes mentor and protégé matching, speed mentoring open to all attendees, a career fair, an alumni networking session, and a student volunteer program that provides travel and registration support in exchange for helping run the conference. It also has an Early Career Program for those who have already graduated or recently started careers.

The volunteer route is the answer to "I cannot afford to go." Student volunteering at SC comes with support for registration and travel. Volunteering can also be even better than attending, because you spend the week working alongside the people you are trying to meet rather than sitting in sessions next to them. Be sure to watch for the volunteer application windows in February and March for each year's conference.

Everything else

- **Regional research and education networks.** Most states or regions have one, and they run meetings that are smaller, cheaper, and easier to talk to people at than a national conference.
- **Vendor webinars and meetups.** Hardware and software companies in this space run frequent free events. Less community, more technology, but a legitimate way to learn and to meet people who move between industry and academia.
- **Regional research software engineering groups.** Strong in both academia and industry, and a good route in if software is your angle.
- **Your own campus.** Email the research computing group. This remains the single highest-return action available to most readers, and almost nobody does it.

The thing that actually builds a reputation

Attending gets you information. Contributing gets you known.

Volunteer for a conference committee. Write a poster or a short paper. Give a lightning talk. Help organize a local meetup. Answer questions on Ask.CI. Take a small role in a CaRCC working group.

The reason this works is specific and worth understanding: it changes how people categorize you. Someone who attends is an attendee. Someone who does something visible is a person who contributes to the community, and that is what hiring managers in a small field are looking for when they think about who to call. You do not need seniority to do any of it.

Networking also pays off in ways unrelated to job hunting. It is how you find out how another institution solved the problem you are stuck on, and how you learn the politics of a place before you take a job there.

Finding a mentor

Both conferences above run formal mentor matching, and so does the national ACCESS mentoring program described in Section Three. Those are the easiest possible entry points, because someone else does the introducing. Sign up.

Outside those programs, the approach that works is small and specific. Do not ask someone to be your mentor. Ask them one concrete question, or ask for fifteen minutes about how they got into the work. People in this field say yes to that at a rate that surprises newcomers. Follow up once with what you did about their advice, and you have started a relationship rather than made a request.

Reading "other duties as assigned"

Nearly every RCD posting contains a long skills list and some version of this phrase. It alarms newcomers, who read it as either a trap or a sign they are underqualified. Neither is usually right.

Here is what is actually going on. These teams are small and cover an enormous range. A group of five people may be responsible for systems, storage, security, training, consulting, and software support across an entire university. Work gets distributed to whoever can take it. The long list describes the team's scope, not one person's required expertise, and the phrase is honest rather than sinister.

So apply anyway, and lead with your strongest attributes rather than trying to address every bullet.

But do ask about it, because there is a real difference between healthy breadth and a job with no boundaries. Good questions: How has this role changed over the last few years? How do new responsibilities get added, and who decides? What did the last person in this position spend most of their time on?

A group that has thought about this will answer easily. A group that has not will tell you something useful in how it struggles to answer.

Questions worth asking in an interview

Interviews in this field are usually two-way to an unusual degree, because good candidates are scarce and hiring managers know it. Asking substantive questions helps you and signals seriousness.

About the work itself

- Walk me through a typical week, and a typical month.
- Who will I work with most closely, and what do they do?
- What will I actually be responsible for in the first year?
- Does this role include software development, or not? (Postings are often ambiguous, and the answer changes the job entirely.)
- What are the on-call expectations, if any?

About whether you would succeed

- What is the onboarding process, and what are the goals for the first ninety days?
- How are people evaluated in this role, and how often?
- What does success look like a year in?
- How does the group support people in learning new skills?
- How does the team handle documentation and project management?

About the people and the place

- What is the best part of your job? What part is frustrating or boring?
- How has this group changed over the past few years, and where do you see it going?
- What is the culture like here?

About stability

- Is this position funded from the operating budget or from a grant, and what happens when the grant ends? Section Six explains why this matters.

The questions that need careful wording. Some things you legitimately need to know can land badly if asked bluntly, because they read as a lack of commitment. Ask about the practice rather than your own intentions.

Instead of	Ask
What is the work-life balance like?	How does the team handle busy periods and system outages? What does the group do to keep the load sustainable?
Can I work remotely?	How does the team currently work, in terms of onsite and remote, and how has that evolved?
How do you prevent scope creep?	How has this role's scope changed over time, and how does the team decide what to take on?
Is this a nice place to live?	What do people on the team like about being based here?

Interviewers who have thought about their own team will answer all of these openly. Interviewers who bristle at any of them have told you something worth knowing.

One caution about using AI in this process

People use AI tools to prepare for interviews and to write applications, and if used well that is fine. Two things to keep in mind.

Hiring managers in this field notice generated application material, partly because they read a great deal of it and partly because these are jobs that select for the ability to communicate specifically about concrete problems. A cover letter full of confident generalities reads exactly like the vague job postings this guide warns you about.

More importantly, a large part of what an interview is testing is how you think about a problem you have not seen before, out loud, in real time. That is the actual job. Practicing with a tool is useful. Outsourcing your answers is not, and it will surface in the first situational question. Many institutions have licenses for AI tools, meaning that researchers will often come for help after already trying and failing to get AI to solve their problem. If you cannot offer value beyond typing an AI prompt, you will not be able to properly support the researchers.

How networking and hiring connect, plainly

Being visible in the community means you hear about openings before they are posted, and sometimes before they exist. It means someone can vouch for you, which matters disproportionately in a small field. It means you arrive at an interview already knowing something about the group's culture and politics. And it means that when a hiring manager has funding and a vague sense of needing someone, your name is available to be thought of.

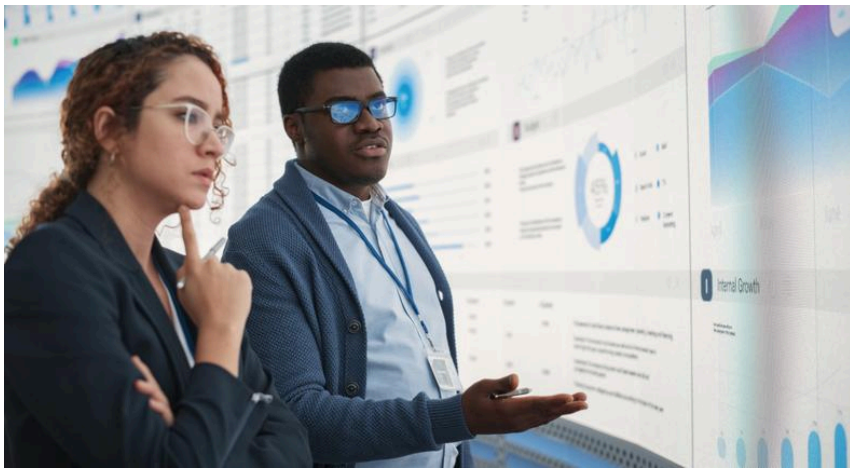
None of this requires being extroverted or well connected already. It requires showing up somewhere free, repeatedly, and eventually doing something small and visible.

Section Eight: Where to Start

If you have read this far, you now know something most people never learn: that there is an entire profession standing behind modern research, that it has a name, and that the people in it mostly arrived by accident.

That last part is the part worth holding onto. Nobody in this field followed a prescribed path, because there is not one. The physicist who now runs a supercomputing center, the librarian who became a data specialist, the help desk technician who taught themselves Linux and never looked back, the student who took a part-time job racking servers and stayed twenty years. None of them qualified in advance. They got interested, got close to the work, and learned the rest on the job.

The barriers that feel largest from outside turn out to be the smallest. You do not need a specific degree or certification. You do not need to already know everything, and the people hiring do not expect you to. What the field is short of is people who listen well, keep going when a problem is unusual or difficult, and care that someone else's work succeeds.



Every RCD career starts with a single first step: a question, an email, or a free training session. Curiosity is the only requirement for getting started.

What you do need is to get closer. This guide cannot do that part.

Three things you can do this month, all free:

1. **Join the CaRCC People Network** and sit in on a track call. Listen. That is enough for the first one.
2. **Email your campus research computing group,** or the nearest one, and ask if someone would spend fifteen minutes telling you what they do. They will say yes more often than you expect. You do not need to know Linux, or any other technical skill, before you reach out. Saying you are curious and just getting started is enough.
3. **Pick one free training program** from Section Three or Four and sign up for the next session.

Any one of them puts you in the room. All three, and you will know within a couple of months whether this work is for you.

One last thing. The people who do this work are rarely named in the paper, the press release, or the news story about the discovery. That invisibility is real, and it is worth deciding early whether you can live with it. Most people in this field can, for a specific reason: they know exactly which discoveries would not have happened without them, and so do the researchers.

If that sounds like enough, welcome. Come find us.

Appendix A: References and Resources

Professional communities and networks

- ACM SIGHPC Systems Professionals. (n.d.). *ACM SIGHPC Systems Professionals Chapter*. https://sighpc-syspros.org/
- Ask.CI. (n.d.). *Ask.CI: Community question-and-answer site for research computing and cyberinfrastructure*. https://ask.cyberinfrastructure.org/
- Campus Champions. (n.d.). *Campus Champions*. https://campuschampions.cyberinfrastructure.org/
  Roughly 800 members at more than 300 institutions.
- Campus Research Computing Consortium. (n.d.). *CaRCC: Campus Research Computing Consortium*. https://carcc.org/
  Free to join.
- Campus Research Computing Consortium. (n.d.). *CaRCC People Network*. https://carcc.org/people-network/
  More than 2,300 members from over 400 institutions, with monthly track calls, interest groups, and an active job-posting stream.
- CCMNet. (n.d.). *CCMNet Mentoring* [Affinity group]. ACCESS Support. https://support.access-ci.org/node/5951 *Cyberinfrastructure Community-wide Mentorship Network; peer mentoring for cyberinfrastructure professionals.*
- Digital Research Alliance of Canada. (n.d.). *Digital Research Alliance of Canada*. https://alliancecan.ca/
- Internet2 Research Engagement. (n.d.). *RCD Ecosystem*. Internet2. https://spaces.at.internet2.edu/display/RE/RCD+Ecosystem *A map of organizations in the field.*
- Research Data Access and Preservation Association. (n.d.). *Research Data Access and Preservation Association*. https://rdapassociation.org/
- U.S. Research Software Engineer Association. (n.d.). *US-RSE*. https://us-rse.org/

Training programs

- Certified Cyberinfrastructure Facilitator Training and Development. (2025, August 26). *Birds of a feather: CCIFTD: Certified Cyberinfrastructure Facilitator Training & Development* [Conference session description]. Zenodo. https://zenodo.org/records/16729614 *Non-matriculated certification of professional development for CI facilitators; NSF Award OAC-2118193.*
- CyberAmbassadors. (n.d.). *CyberAmbassadors*. Michigan State University. https://sites.google.com/msu.edu/cyberambassadors/ *NSF Award 1730137.*
- LinkedIn. (n.d.). *LinkedIn Learning*. https://www.linkedin.com/learning/
- Linux Clusters Institute. (n.d.). *Linux Clusters Institute workshops*. https://linuxclustersinstitute.org/
- The Carpentries. (n.d.). *The Carpentries*. https://carpentries.org/ *Software, Data, and Library Carpentry.*
- The Carpentries. (n.d.). *Carpentries workshops*. https://carpentries.org/workshops/

- The Turing Way Community. (n.d.). *The Turing Way: A handbook for reproducible, ethical and collaborative research*. https://book.the-turing-way.org/
- University of Oklahoma. (n.d.). *Virtual Residency Program*. OU Supercomputing Center for Education & Research. http://www.oscer.ou.edu/virtualresidency/ *Supported by the National Science Foundation.*
- Visionlearning. (n.d.). *Process of science*. https://www.visionlearning.com/en/library/Process-of-Science/49 *Free modules on the process and practice of science.*

Computing resources for students and new users

- NSF ACCESS. (n.d.). *ACCESS: Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support*. https://access-ci.org/ *Free allocations on national supercomputers for U.S. researchers and students, including ACCESS MATCH consultation and mentoring services.*

Conferences and competitions

- ISC High Performance. (n.d.). *ISC Student Cluster Competition*. https://isc-hpc.com/ *For the European events.*
- Practice and Experience in Advanced Research Computing. (n.d.). *PEARC*. Association for Computing Machinery. https://pearc.acm.org/ *Includes a student program.*
- Student Cluster Competition. (n.d.). *Student Cluster Competition*. https://www.studentclustercompetition.us/ *Includes eligibility for high school and undergraduate students and the lower-barrier virtual companion competition.*
- The International Conference for High Performance Computing, Networking, Storage, and Analysis. (n.d.). *SC conference and Students@SC program*. https://supercomputing.org/

Job boards and listings

Ask.CI. (n.d.). *Jobs*. https://ask.cyberinfrastructure.org/c/jobs/62

Code4Lib. (n.d.). *Code4Lib jobs*. https://jobs.code4lib.org/

Computing Research Association. (n.d.). *CRA career center*. https://careercenter.cra.org/

EDUCAUSE. (n.d.). *EDUCAUSE jobs*. https://jobs.educause.edu/

HigherEdJobs. (n.d.). *HigherEdJobs*. https://www.higheredjobs.com/

HPCwire. (n.d.). *HPCwire jobs*. https://jobs.hpcwire.com/

Inside Higher Ed. (n.d.). *Inside Higher Ed Careers*. https://careers.insidehighered.com/

jobRxiv. (n.d.). *jobRxiv: Research jobs across the sciences*. https://jobrxiv.org/

National Science Foundation. (n.d.). *Research Experiences for Undergraduates (REU)*. https://www.nsf.gov/funding/initiatives/reu

The Chronicle of Higher Education. (n.d.). *Chronicle Jobs*. https://jobs.chronicle.com/

U.S. Department of Energy. (n.d.). *Jobs at the National Labs*. https://www.energy.gov/jobs-national-labs

U.S. Research Software Engineer Association. (n.d.). *US-RSE job board*. https://us-rse.org/jobs/

Women in High Performance Computing. (n.d.). *Women in HPC*. https://womeninhpc.org/

Career frameworks and job descriptions

- Campus Research Computing Consortium. (n.d.). *CaRCC facings framework*. https://carcc.org/rcd-professionalization/facings/
- Campus Research Computing Consortium. (n.d.). *CaRCC HR Job Family Matrix*. https://carcc.org/products_resources/hrjf-matrix/ *Levels from entry to expert, responsibilities, and salary ranges.*
- Campus Research Computing Consortium. (n.d.). *RCD Career Arcs working group*. https://carcc.org/career-arcs/
- Campus Research Computing Consortium. (n.d.). *RCD Nexus*. https://carcc.org/rcd-nexus/ *The community's career and resource center.*
- RCD Professionalization Working Group. (2018). *Research computing and data professionals job elements and career guide*. Zenodo. https://doi.org/10.5281/zenodo.7108947
- U.S. Research Software Engineer Association. (n.d.). *RSE roles*. https://us-rse.org/rse-roles/ *Job description database, useful for seeing how these roles are written in practice.*

Research and reports cited

- ALERTCalifornia. (n.d.). *ALERTCalifornia*. University of California San Diego. https://alertcalifornia.org/
- Campus Research Computing Consortium. (2022, September 12). *Who we (RCD professionals) are*. https://carcc.org/2022/09/12/who-we-rcd-professionals-are/ *2021 workforce survey.*
- California Governor's Office. (2023, October 24). *TIME recognizes CAL FIRE AI & AlertCalifornia as a Best Invention of* 2023 [Press release]. https://www.gov.ca.gov/2023/10/24/time-recognizes-cal-fire-ai-alertcalifornia-as-a-best-invention-of-2023/ *Reproduces the TIME entry.*
- Chaudhry, S., Pazouki, A., Schmitz, P., Hillery, E., & Kee, K. (2022). Understanding factors that influence research computing and data careers. In *Practice and Experience in Advanced Research Computing (PEARC '22)* (Article 25, pp. 1-9). Association for Computing Machinery. https://doi.org/10.1145/3491418.3530292

- Maimone, C., Brown, C., Grash, K., Reidy, C., & Stauffer, A. (2023). Compensation of academic research computing and data professionals. In *Practice and Experience in Advanced Research Computing (PEARC '23)* (pp. 145-152). Association for Computing Machinery. https://doi.org/10.1145/3569951.3593599
- Maimone, C., Yockel, S., Middelkoop, T., Stauffer, A., & Reidy, C. (2022). Characterizing the US research computing and data (RCD) workforce. In *Practice and Experience in Advanced Research Computing (PEARC '22)* (pp. 1-7). Association for Computing Machinery. https://doi.org/10.1145/3491418.3530289 *Based on 563 survey responses.*
- Middelkoop, T., Mizumoto, C., Strachan, S., Balamurugan, D., Delinger, S., Fosso Tande, J., McCaffrey, D., Myatt James, A., Reddy, D., & Schmitz, P. (2023). Career phases in research computing and data. In *Practice and Experience in Advanced Research Computing (PEARC '23)* (pp. 374-377). Association for Computing Machinery. https://doi.org/10.1145/3569951.3597562
- National Center for Science and Engineering Statistics. (2025). *Higher Education Research and Development Survey: Fiscal year 2024*. National Science Foundation. https://ncses.nsf.gov/surveys/higher-education-research-development/2024
- National Science and Technology Council, Subcommittee on Research Security. (2022, January). *Guidance for implementing National Security Presidential Memorandum 33 (NSPM-33) on national security strategy for United States government-supported research and development*. Office of Science and Technology Policy. https://www.whitehouse.gov/wp-content/uploads/2022/01/010422-NSPM-33-Implementation-Guidance.pdf
- Smith, P. M. (2024). The value proposition of campus high performance computing facilities to institutional productivity: A production function model. *SN Computer Science, 5*(5), Article 551. https://doi.org/10.1007/s42979-024-02888-0
- Smith, P. M., Gemmill, J., Hancock, D. Y., O'Shea, B. W., Snapp-Childs, W., & Wilgenbusch, J. (2025). Application of the cyberinfrastructure production function model to R1 institutions. *Frontiers in Research Metrics and Analytics, 10*, Article 1449996. https://doi.org/10.3389/frma.2025.1449996
- Stauffer, A. (2026, March 28). *2025 research computing and data (RCD) workforce survey* [Survey instrument]. Zenodo. https://doi.org/10.5281/zenodo.19299524
- TIME. (2023, October 24). The best inventions of 2023: Stopping wildfires: AlertCalifornia and Cal Fire AI wildfire detector. https://time.com/collection/best-inventions-2023

Appendix B: Acknowledgements

The RCD Nexus Day workshop series is hosted by the Campus Research Computing Consortium (CaRCC), with generous support from Internet2. The following individuals led and participated in the workshop that generated the content for this guide, or served as content reviewers following the workshop:

Workshop organizers & facilitators:

- Dana Brunson, Internet2
- Stephen Deems, Internet2
- Levi Dolan, Yale University
- Bob Freeman, Harvard University
- Betsy Hillery, Purdue University
- Daphne McCanse, Internet2
- Lauren Michael, Internet2
- Timothy Middelkoop, Internet2
- Winona Snapp-Childs, Indiana University
- Ashley Stauffer, Penn State University
- Michael Weiner, Georgia Institute of Technology

Workshop attendees, content contributors and reviewers:

- James Afful, Iowa State University
- Ahmad Al-Omari, Our Lady of the Lake University
- Izzat Alsmadi, Texas A&M San Antonio
- Anas AlSobeh, Utah Valley University
- Gladys Andino, University of Virginia
- Sahitya Avadhanam, University of Minnesota
- Munkhtuya Aziz, University of Minnesota
- Marcos Botto Tornielli, University of Delaware
- Renée F. Brown, Arizona State University
- Katia Bulekova, Boston University
- Cyd Burrows-Schilling, University of California San Diego
- James Cannon, University of Michigan
- Sarvani Chadalapaka, University of California Merced
- Thomas Cheatham, University of Utah
- Eric Collins, Mississippi State University
- Sharon Colson, Tennessee Tech University
- Moira Downey, NC State University
- Matthew Economou, Research Data and Communication Technologies
- Matthew Esposito, Clemson University
- Satyar Foroughi, University of Central Florida
- Jacob Fosso Tande, North Carolina State University
- Toolika Ghose, University of Nevada Las Vegas
- David Hart, Internet2

- Seth Herron, University of California, Berkeley
- Curt Hillegas, Princeton University
- Daniel Howard, National Center for Atmospheric Research
- Aaron Jezghani, Georgia Institute of Technology
- Marina Kraeva, Iowa State University
- Mohamed Labadi, University of Chlef
- John Michael Lowe, Indiana University
- Dennis Milechin, Boston University
- Jim Moroney, Tennessee Technological University
- Krishna Chaitanya Mummadi, University of Maryland Baltimore
- Manish Nalluri, Oregon Institute of Technology
- Tannistha Nandi, University of British Columbia
- Sai Vinay Nandigam, Clemson University
- Henry Neeman, University of Oklahoma
- Andres Ortiz Almazo, Temple University
- Rion Parsons, University of California, Santa Cruz
- Chuck Pavloski, University of Illinois
- Xinlei Qiu, Stanford University
- Bryan Raney, Rice University
- Krishna Rauniyar, University of Louisiana at Lafayette
- David Reddy, University of South Carolina
- Chris Reidy, University of Arizona
- Louis Roseguo, University of Hawai'i System
- Jasmine Sahota, Mississippi State University
- Semir Sarajlic, Vanderbilt University
- Swabir Silayi, George Mason University
- Jason Simms, Swarthmore College
- Raminder Singh, Harvard University
- Mary Ellen Sloane, Middle Tennessee State University
- Esakkiammal Subbukutty, Nirma University
- Nandan Tandon, University of Central Florida
- Olamide Tawose, Lincoln University of Pennsylvania
- Uku-Kaspar Uustalu, Tufts University
- Kevin Van Slyke, University at Buffalo (SUNY)
- Wenyu Wang, The Ohio State University
- Tony Weaver, Franklin & Marshall College
- Scott Yockel, Google Cloud Public Sector